



400-650-7088

TongLINK/Q-CN V10.0

灾备及故障转移手册

版本: 10041A01

2025 年 9 月

声明

版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech** 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

本手册版本（10041A01）适用于TongLINK/Q-CN V10.0.4.1。

文档版本	适用产品版本	更新内容	更新日期
10041A01	V10.0.4.1	更新如下内容： 更新版权声明。	2025/9/23
10040A01	V10.0.4.0	TongLINK/Q-CN V10.0.4第一次正式发布。	2025/5/23

前言

本文档是TongLINK/Q-CN V10.0产品的用户使用手册之一，介绍使用灾备及故障转移的相关概念、配置、参数及示例。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

• 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

• 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

• 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

1. 掌握Linux系统基本操作；
2. 掌握K8s的搭建和使用；
3. 熟悉Java开发语言。

• 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

• 实践经验

为了最大程度地受益于本文档，建议您具备一定实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

用户使用手册集

TongLINK/Q-CN V10.0为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongLINK/Q-CN_V10.0用户手册导读	提供手册集的目录及手册版本说明。
2	001_TongLINK/Q-CN_V10.0快速使用手册	提供产品介绍，在K8s和虚拟机上安装部署产品的基本步骤及验证方式。

编号	手册集文档	说明
3	002_TongLINK/Q-CN_V10.0配置参数手册	提供配置文件及参数的详细说明。
4	003_TongLINK/Q-CN_V10.0监控管理手册	提供在K8s和虚拟机上安装部署监控管理组件的基本步骤及验证方式。
5	004_TongLINK/Q-CN_V10.0管理控制台使用手册	详细介绍管理控制台的集群管理、集群监控、告警管理和用户管理相关功能的使用说明。
6	005_TongLINK/Q-CN_V10.0命令行使用手册	详细提供产品中主要命令行的作用、相关参数及使用示例。
7	006_TongLINK/Q-CN_V10.0Java客户端开发手册	介绍使用Java客户端生产消费消息的相关概念、配置、参数及示例。
8	007_TongLINK/Q-CN_V10.0Kafka协议部署手册	提供在K8s和虚拟机上安装部署Kafka协议处理程序的基本步骤及验证方式。
9	008_TongLINK/Q-CN_V10.0AMQP协议部署手册	提供在K8s和虚拟机上安装部署AMQP协议处理程序的基本步骤及验证方式。
10	009_TongLINK/Q-CN_V10.0JMS客户端开发手册	介绍如何将适配ActiveMQ的JMS客户端迁移为适配TongLINK/Q-CN的JMS客户端。
11	010_TongLINK/Q-CN_V10.0MQTT协议部署手册	提供在K8s和虚拟机上安装部署MQTT协议处理程序的基本步骤及验证方式。
12	011_TongLINK/Q-CN_V10.0服务端进阶配置指南	提供在K8s和虚拟机上开启TongLINK/Q-CN的国密通信、消息轨迹、消息过滤等高级功能特性的方法。
13	012_TongLINK/Q-CN_V10.0灾备及故障转移手册	介绍使用灾备及故障转移的相关概念、配置、参数及示例。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：www.tongtech.com
- 电话：400-650-7088
- 邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

- 您的姓名

- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

目录

声明	1
版本变更说明	2
前言	3
1. 简介	7
2. Client端集群级故障转移	8
2.1. 客户端常规配置	8
2.2. 集群级故障转移简介	10
2.2.1. 使用场景	10
2.3. 自动集群级故障转移	10
2.3.1. 概念	10
2.3.2. 触发条件	11
2.3.3. 工作原理	11
2.3.4. 使用方法	12
2.4. 手工集群级故障转移	14
2.4.1. 概念	14
2.4.2. 触发条件	15
2.4.3. 工作原理	15
2.4.4. 使用方法	16
2.5. 优势与局限性	19
2.5.1. 优势	19
2.5.2. 局限性	19
2.6. 集群级故障转移失败原因	19
2.7. 集群级故障转移和异地复制的区别	19
3. GEO	21
3.1. 功能简介	21
3.2. 使用场景	21
3.3. 配置方法	21
3.4. 业务验证	22
4. 订阅同步	26
4.1. 功能简介	26
4.2. 使用场景	26
4.3. 配置方法	26
4.4. 业务验证	26
4.5. 局限性	30
附录 A: 术语说明	31

1. 简介

TongLINK/Q-CN的灾备方案基于其分布式架构和跨地域复制能力，结合多副本机制与智能切换策略，可构建高可用、强一致性的容灾体系。

TongLINK/Q-CN除集群内部的多副本，Quorum算法，机架感知，地域感知等，保障分布式消息队列的强一致性和容错能力外，还提供如下集群间的灾备方案：

- **Client端集群级故障转移**

可以在客户端配置多个集群，故障情况下客户端实现自动或者手动连接到备用集群。

- **跨地域复制（Geo-Replication，简称GEO复制）**

通过BookKeeper集群和Broker协调实现跨地域数据同步，支持消息在多个地域间自动复制，支持按照Topic，按照Namespace级别进行复制，确保数据冗余存储。

通过GEO复制可以实现多种方式的组网，如：双主/三主，主备，两地三中心，多中心汇聚等。

- **多区域大集群**

多地域多节点可以组成一个大集群，根据地域感知，机架感知方式，多副本分布到不同的地区或机架，实现每个租户指定使用节点物理隔离。

2. Client端集群级故障转移

2.1. 客户端常规配置

可以仅使用TongLINK/Q-CN集群的一个URL来实例化PulsarClient对象，如下所示：

```
PulsarClient client = PulsarClient.builder()

    .serviceUrl("pulsar://localhost:6650")

    .build();
```

若存在多个代理，也可以使用该方式启动一个PulsarClient，如下所示：

```
PulsarClient client = PulsarClient.builder()

    .serviceUrl("pulsar://localhost:6650,localhost:6651,localhost:6652")

    .build();
```

如果创建客户端，则可以使用loadConf配置。在loadConf中可使用以下参数配置。

参数名	类型	描述	默认
serviceUrl	String	TongLINK/Q-CN集群地址。	-
authPluginClassName	String	身份验证插件的名称。	-
authParams	String	身份验证插件的参数。 示例： key1:val1,key2:val2	-
operationTimeoutMs	long	操作超时时间，单位秒。 在此时间段内生产者创建、订阅和取消订阅操作会一直重试，若超过这个时间还没成功将会被标记失败。	30
statsIntervalSeconds	long	每次统计信息之间的间隔，单位秒。 Stats被激活，statsInterval为正值； statsIntervalSeconds至少设置为1秒。	60
numIoThreads	int	用于处理与broker连接的线程数。	1

参数名	类型	描述	默认
numListenerThreads	int	用于处理消息监听器的线程数。 监听器线程池在所有使用“监听器”模型获取消息的消费者和读者之间共享。对于一个给定的消费者，监听器始终从同一线程调用，以确保消息排序。 如果需要多个线程处理单个主题，则要创建一个共享订阅和这个订阅的多个消费者，但无法确保消息有序。	1
useTcpNoDelay	boolean	是否在TCP连接上使用无延迟标志来禁用Nagle算法。	True
enableTls	boolean	是否在连接上使用TLS加密。 注意：此参数已*弃用*。如果要启用TLS，请改用pulsar+ssl://in serviceUrl。	False
tlsTrustCertsFilePath	String	受信任的TLS证书文件的路径。	false
tlsAllowInsecureConnection	boolean	客户端是否接受来自broker的不受信任的TLS证书。	False
tlsHostnameVerificationEnable	boolean	是否启用TLS主机名验证。	False
concurrentLookupRequest	int	每个broker连接上允许发送的并发查找请求数，防止broker过载。	5000
maxLookupRequest	int	每个broker连接上允许的最大查找请求数，防止broker过载。	50000
maxNumberOfRejectedRequestPerConnection	int	当前连接关闭后，客户端创建新连接连接不同broker后一定时间范围内（60秒）broker拒绝请求的最大次数。	50
keepAliveIntervalSeconds	int	每个客户端与broker的连接保持活跃的时间，单位秒。	30
connectionTimeoutMs	int	客户端与broker建立连接的超时时间，单位毫秒。	10000
requestTimeoutMs	int	完成请求的最长持续时间，单位毫秒。	60000

参数名	类型	描述	默认
defaultBackoffIntervalNanos	int	默认重试间隔时间，单位纳秒。	TimeUnit.MILLISECONDS.toNanos(100);
maxBackoffIntervalNanos	长	重试间隔的最长持续时间，单位纳秒。	TimeUnit.SECONDS.toNanos(30)
socks5ProxyAddress	SocketAddress	SOCKS5代理地址。	-
socks5ProxyUsername	String	SOCKS5代理用户名。	-
socks5ProxyPassword	String	SOCKS5代理密码。	-

2.2. 集群级故障转移简介

本章介绍集群级故障转移的概念、优势、用例、约束、用法、工作原理以及更多信息。

根据故障转移的类型分为**自动集群级故障转移**和**手动集群级故障转移**。

2.2.1. 使用场景

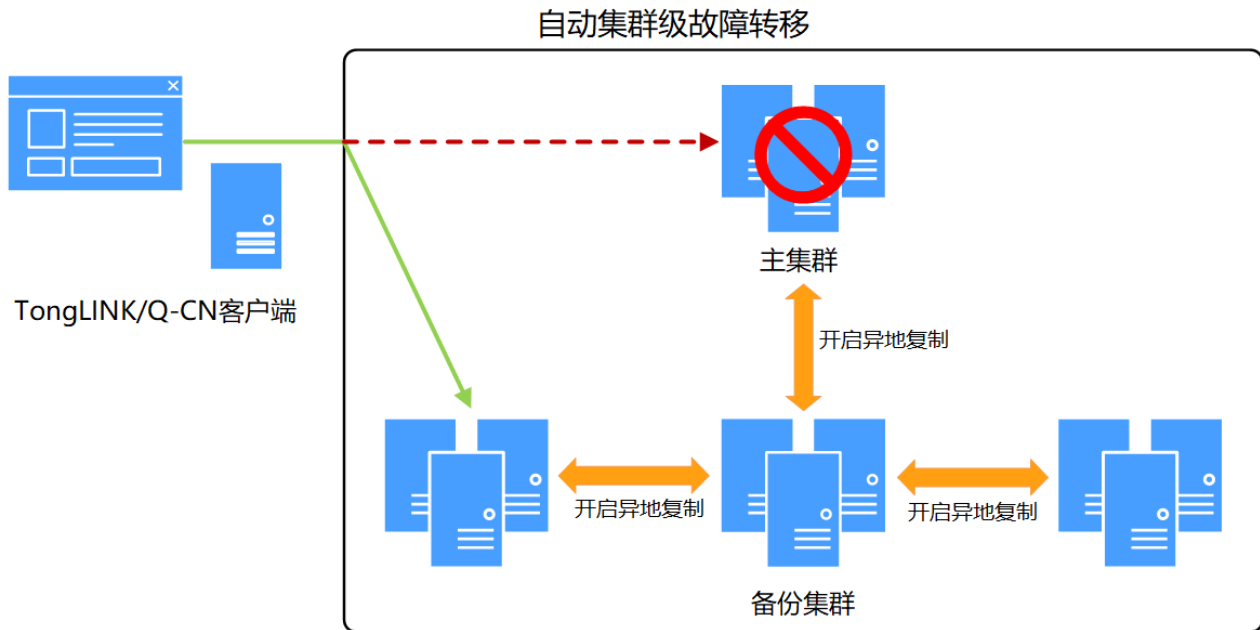
集群级故障转移以多种方式保护您的环境，包括但不限于：

- 灾难恢复：集群级故障转移可以自动/手动无缝地将一个主集群上的生产工作负载转移到一个或多个备份集群，从而确保最小的数据丢失并减少恢复时间。
- 计划迁移：如果要将生产工作负载从旧集群迁移到新集群，可以通过集群级故障转移来提高迁移效率。例如，在发生故障转移事件时测试数据迁移是否顺利，在迁移前识别可能存在的问题和风险。

2.3. 自动集群级故障转移

2.3.1. 概念

自动集群级故障转移支持客户端在检测到故障转移事件时，根据用户设置的配置检测策略自动无缝地从连接主集群切换到连接一个或多个备份集群。如下图所示：



2.3.2. 触发条件

当客户端在配置时间内无法连接到主集群时，会触发自动集群级故障转移。

导致客户端长时间无法连接到主集群的原因包括但不限于：

- 网络故障：互联网连接丢失。
- 断电：主集群关机时间超限。
- 业务错误：主集群出现错误（如主集群受时间限制无法正常工作）。
- 存储空间崩溃：主集群存储空间不足，但备份服务器上对应的存储空间正常。

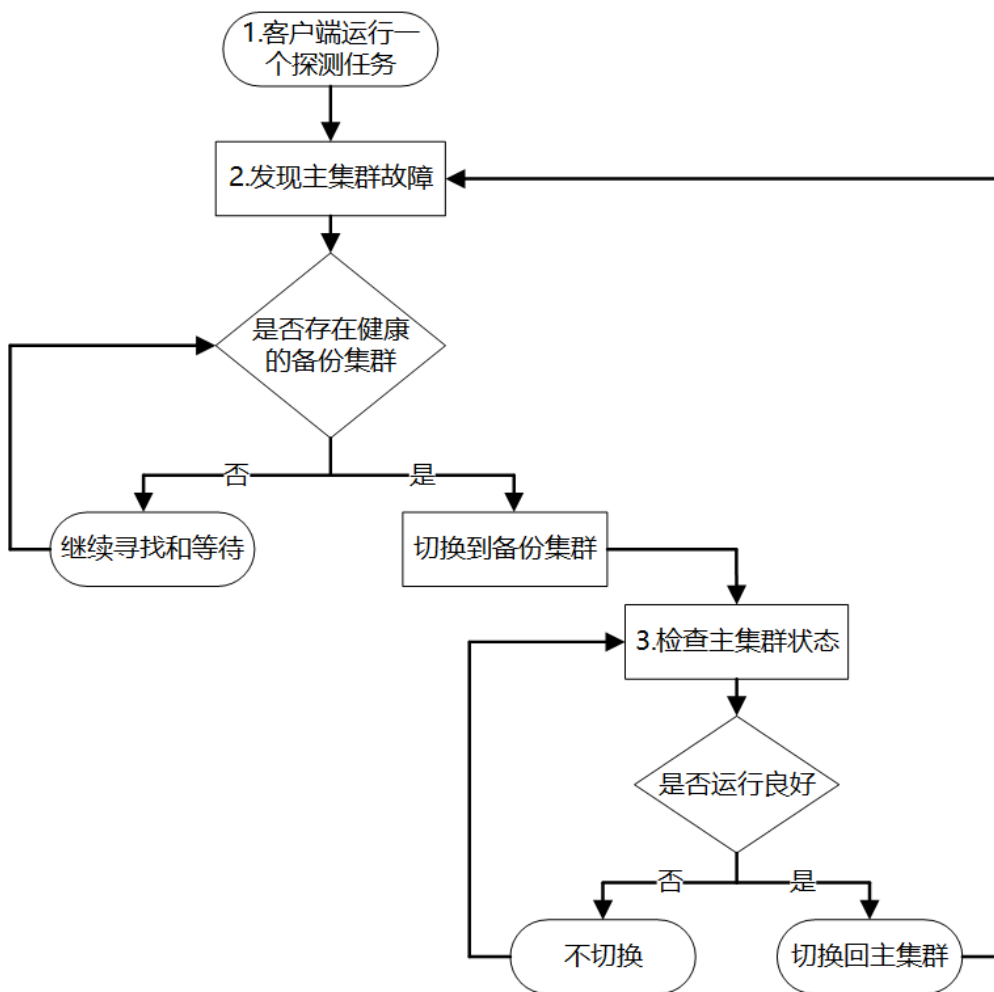
2.3.3. 工作原理

在自动故障转移集群中，主集群和备份集群知道彼此的可用性。自动故障转移集群无需管理员干预即可执行以下操作：

1. 客户端以checkInterval中定义的探测时间间隔运行一个探测任务。
2. 如果探测任务发现主集群的故障时间超过failoverDelay参数中设置的时间，它会在备份集群中搜索可用的健康集群。
 - a. 如果有健康的备份集群，客户端按照定义的顺序切换到备份集群secondary。
 - b. 如果没有健康的备份集群，客户端不执行切换，探测任务继续寻找可用的备份集群。
3. 探测任务检查主集群是否运行良好。
 - a. 如果主集群恢复正常，并且持续健康时间超过了switchBackDelay中设置的时间，则客户端切换回主集群。
 - b. 如果主集群没有恢复正常，客户端不执行切换。

流程如下图所示：

自动集群级故障转移工作流程



2.3.4. 使用方法

本章节介绍配置集群级故障转移的步骤。

注意：

- 只有当集群包含足够的资源来处理所有可能的后果时，才应该配置集群级故障转移。备份集群上的工作负载强度可能会显著增加。
- 将集群连接到不间断电源（UPS）设备以降低意外断电的风险。

要求：

- 客户端3.0或更高版本。
- 对于备份集群：
 - BookKeeper节点的数量应大于等于消息分发的副本。
 - ZooKeeper节点数应大于等于3。
- 备份集群已经提前配置有跟主集群一致的租户和命名空间，在broker的配置文件中，topic自动创建打开。否则转移后无法找到对应的租户、命名空间和topic。

如果需要主备集群之间的数据也保持同步，需要满足如下要求：

- 打开主集群和任何从属集群（主到备份或备份到备份）之间的异地复制以防止数据丢失。
- 创建消费者时设置`replicateSubscriptionState`为`true`。

配置：

本节介绍构建使用自动集群级故障转移的Java客户端的示例，即自动触发切换。

```
private PulsarClient getAutoFailoverClient() throws PulsarClientException {

    ServiceUrlProvider failover = AutoClusterFailover.builder()

        .primary("pulsar://localhost:6650")

        .secondary(Collections.singletonList("pulsar://other1:6650","pulsar://other2:6650"))

        .failoverDelay(30, TimeUnit.SECONDS)

        .switchBackDelay(60, TimeUnit.SECONDS)

        .checkInterval(1000, TimeUnit.MILLISECONDS)

        .secondaryTlsTrustCertsFilePath("/path/to/ca.cert.pem")

        .secondaryAuthentication("org.apache.pulsar.client.impl.auth.AuthenticationTls", "tlsCertFile:/path/to/my-
role.cert.pem,tlsKeyFile:/path/to/my-role.key-pk8.pem")

        .build();

    PulsarClient pulsarClient = PulsarClient.builder()

        .build();

    failover.initialize(pulsarClient);

    return pulsarClient;

}
```

配置的参数说明如下：

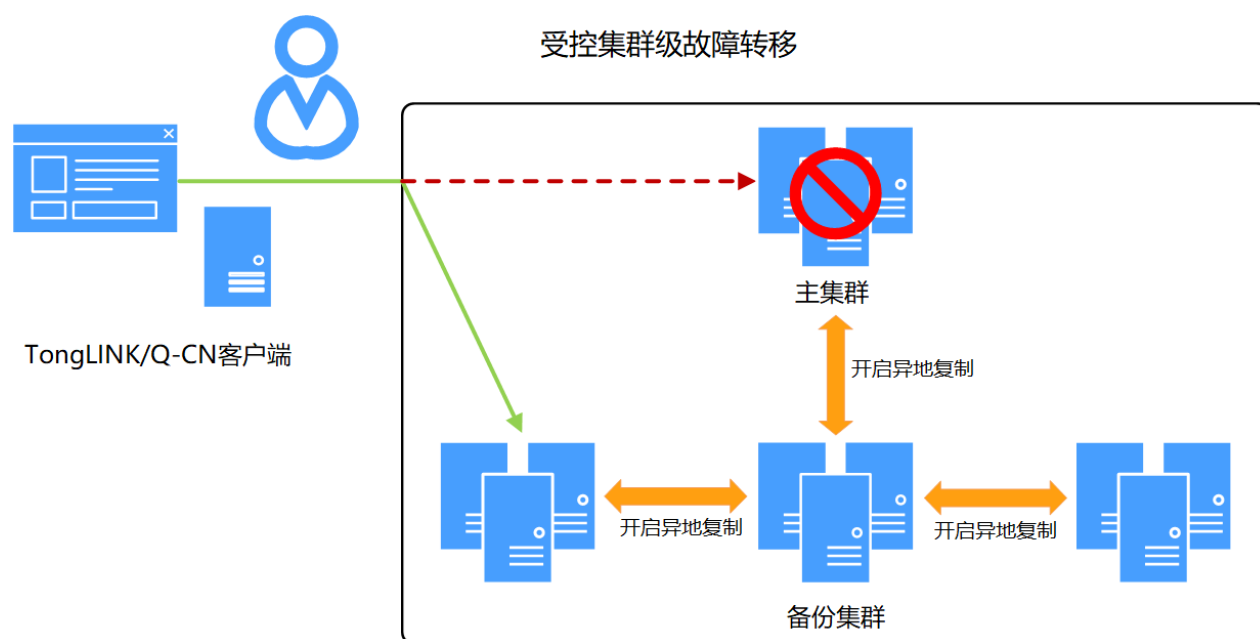
参数	是否必需	描述	默认值
primary	是	主集群的服务URL。	-

参数	是否必需	描述	默认值
secondary	是	一个或多个备份集群的服务URL，多个备份集群使用逗号分隔。 注意： - 按照列表中显示的顺序选择备份集群。 - 如果所有备份集群都可用，客户端选择第一个备份集群。	-
failoverDelay	是	客户端从连接主集群切换到连接备份集群之前的延迟。 自动故障转移由探测任务控制： - 探测任务首先检查主集群的健康状态。 - 如果探测任务发现主集群连续故障时间超过failoverDelayMs，则将TongLINK/Q-CN客户端切换到连接备集群。	-
switchBackDelay	是	TongLINK/Q-CN客户端从连接备份集群切换到连接主集群之前的延迟。 自动故障转移由探测任务控制： - 客户端从主集群切换到备份集群后，探测任务继续检查主集群状态。 - 如果主集群运行良好并且连续保持活跃的时间超过switchBackDelay，则客户端切换回连接主集群。	-
checkInterval	否	执行探测任务的频率（单位：秒）。	30s
secondaryTlsTrustCertificateFilePath	否	备份集群可信任TLS证书文件路径。	-
secondaryAuthentication	否	备份集群的认证。	-

2.4. 手工集群级故障转移

2.4.1. 概念

手工集群级故障转移支持客户端从连接主集群切换到连接一个或多个备份集群。该切换由管理员手动设置。如下图所示：



一旦主集群再次运行，客户端可以自动切换回连接主集群。通常情况下用户不会感知到该过程，用户使用应用程序和服务不会中断或超时。

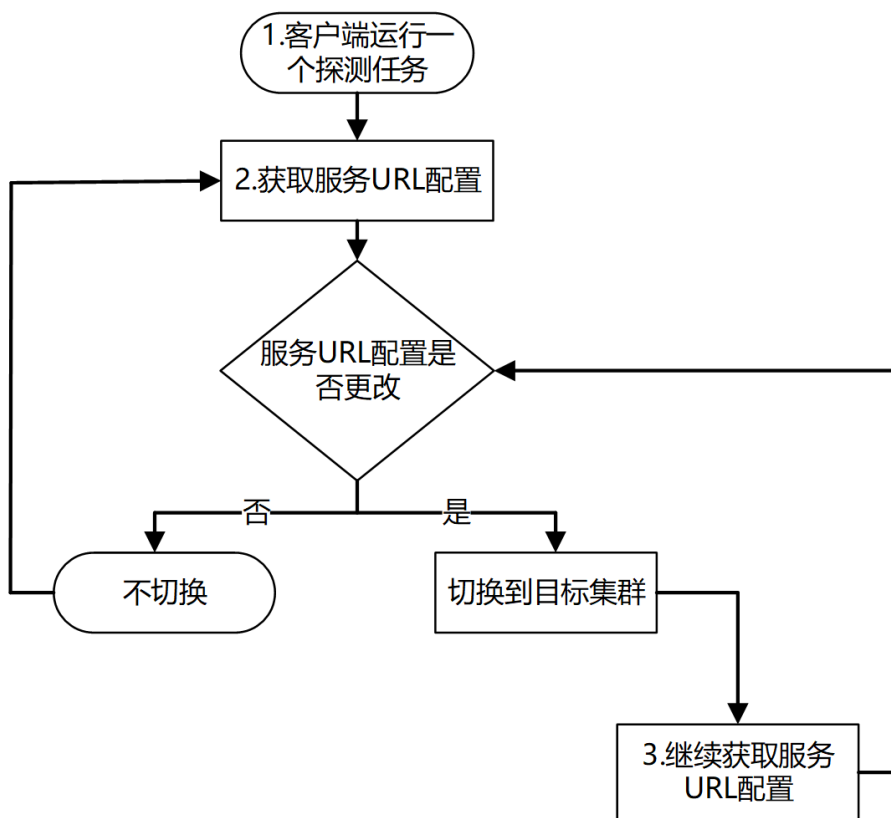
2.4.2. 触发条件

当管理员手动更改UrlProvider的集群配置地址切换时，将触发手工集群级故障转移。

2.4.3. 工作原理

1. 客户端以checkInterval中定义的探测时间间隔运行一个探测任务。
2. 探测任务从URL服务提供程序中获取服务URL配置，该服务由urlProvider配置。
 - a. 如果服务URL配置已更改，则探测任务将切换到目标集群，而不检查目标集群的运行状况。
 - b. 如果服务URL配置未更改，客户端将不执行切换。
3. 如果客户端切换到目标集群，探测任务将继续以checkInterval中定义的间隔从URL服务提供程序中获取服务URL配置。
 - a. 如果服务URL配置已更改，则探测任务将不检查目标集群的运行状况，直接切换到目标集群。
 - b. 如果服务URL配置未更改，则不执行切换。

流程如下图所示：



2.4.4. 使用方法

本章节介绍配置集群级故障转移的步骤。

注意：

- 只有当集群包含足够的资源来处理所有可能的后果时，才应该配置集群级故障转移。备份集群上的工作负载强度可能会显著增加。
- 将集群连接到不间断电源（UPS）设备以降低意外断电的风险。

要求：

- 客户端3.0或更高版本。
- 需要有UrlProvider服务，或者找东方通技术支持。
- 对于备份集群：
 - BookKeeper节点的数量应大于等于消息分发的副本。
 - ZooKeeper节点数应大于等于3。
- 备份集群已经提前配置有跟主集群一致的租户和命名空间，在broker的配置文件中，topic自动创建打开。否则转移后无法找到对应的租户，命名空间和topic。

如果需要主备集群之间的数据也保持同步，需要满足如下要求：

- 打开主集群和任何从属集群（主到备份或备份到备份）之间的异地复制以防止数据丢失。
- 创建消费者时设置`replicateSubscriptionState`为`true`。

配置：

本节介绍构建使用手工集群级故障转移的Java客户端的示例，即切换由管理员手动触发。

注意：可以有一个或多个备份集群，但只能指定一个备份集群。

```
public PulsarClient getControlledFailoverClient() throws IOException {

    Map<String, String> header = new HashMap();

    header.put("service_user_id", "my-user");

    header.put("service_password", "tiger");

    header.put("clusterA", "tokenA");

    header.put("clusterB", "tokenB");

    ServiceUrlProvider provider = ControlledClusterFailover.builder()

        .defaultServiceUrl("pulsar://localhost:6650")

        .checkInterval(1, TimeUnit.MINUTES)

        .urlProvider("http://localhost:8080/test")

        .urlProviderHeader(header)

        .build();

    PulsarClient pulsarClient = PulsarClient.builder().build();

    provider.initialize(pulsarClient);

    return pulsarClient;

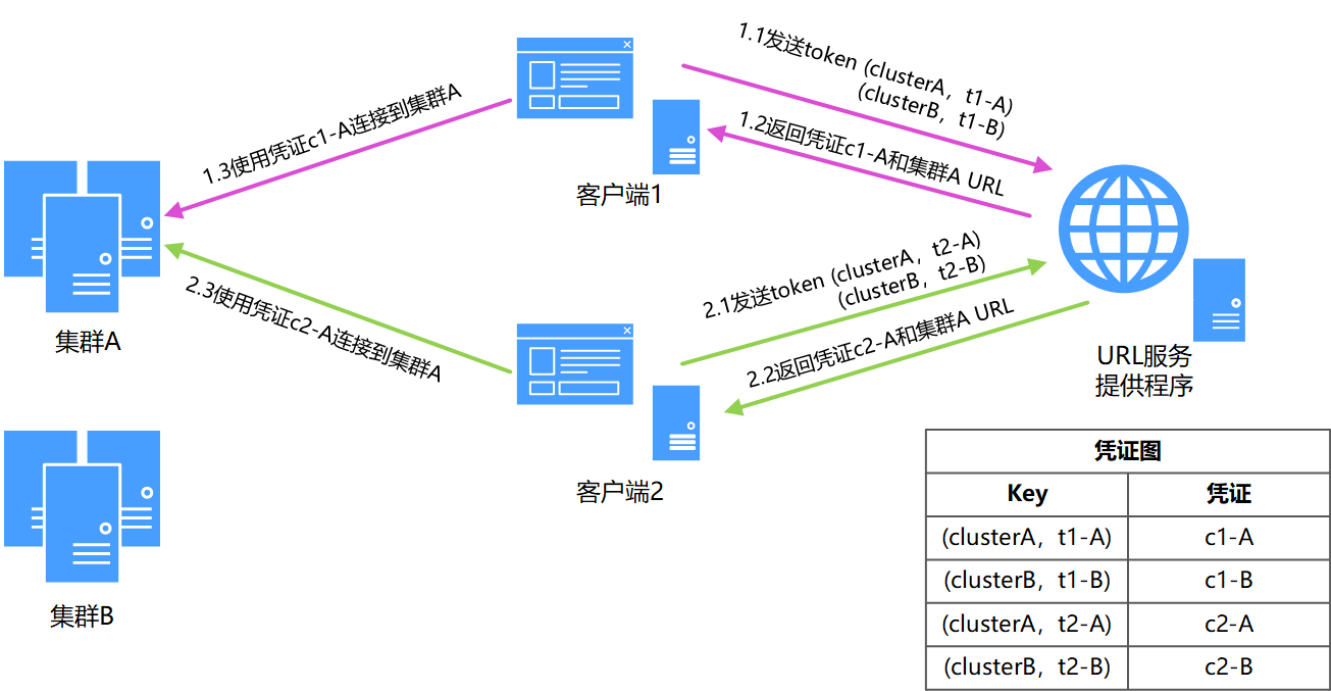
}
```

配置的参数说明如下：

参数	是否必需	描述	默认值
defaultServiceUrl	是	集群地址。	-
checkInterval	否	检查频率。	30s
urlProvider	是	UR提供服务。	-

参数	是否必需	描述	默认值
urlProviderHeader	否	urlProviderHeader是一个包含token和凭证的映射。 如果在客户端与主集群和备份集群之间启用身份验证或授权，则需要提供urlProviderHeader。	-

下图为urlProviderHeader工作原理的示例：



假设要将客户端1连接到集群A。

1. 客户端1向URL服务提供程序发送token t1。
2. URL服务提供程序向客户端返回凭证c1和集群A URL。

URL服务提供程序管理所有token和凭证。它根据不同的token和不同的目标集群URL向不同的客户端返回不同的凭证。

注意：凭证必须在JSON*文件中，并包含如下所示的参数。

```

{

    "serviceUrl": "pulsar+ssl://target:6651",

    "tlsTrustCertsFilePath": "/security/ca.cert.pem",

    "authPluginClassName": "org.apache.pulsar.client.impl.auth.AuthenticationTls",

    "authParamsString": "\"tlsCertFile\": \"security/client.cert.pem\"

    \"tlsKeyFile\": \"security/client-pk8.pem\" \"
    
```

```
}
```

3. 客户端1使用凭证c1连接到集群A。

2.5. 优势与局限性

2.5.1. 优势

集群级故障转移同时提供了容错、持续可用性和高可用性。它带来了许多好处，包括但不限于：

- 降低成本：业务可以自动切换和恢复，不会丢失数据。
- 简化管理：由于不需要用户立即干预，企业可以在“永远在线”的基础上运营。
- 改进的稳定性和稳健性：确保性能持续并最大限度地减少服务停机时间。

2.5.2. 局限性

目前，集群级故障转移可以进行探测以防止数据丢失，但无法检查备份集群的状态。如果备份集群不健康，则无法生产或使用数据。

2.6. 集群级故障转移失败原因

如果活跃的客户端无法访问备份集群，则集群级故障转移会失败。发生这种情况的原因有很多，包括但不限于：

- 断电：备份集群关闭或不能正常工作。
- 存储空间崩溃：主备集群存储空间不足。
- 如果启动故障转移，但由于错误没有集群可以承担可用集群的角色，主集群无法正常提供服务。
- 如果手动发起切换，但无法将服务切换到备用集群服务器，那么系统会尝试将服务切换回主集群。
- 无法在主集群和备份集群之间，或两个备份集群之间进行身份验证或授权。

2.7. 集群级故障转移和异地复制的区别

集群级故障转移是异地复制的扩展，以提高稳定性和稳健性。集群级故障转移依赖于异地复制，二者的区别如下：

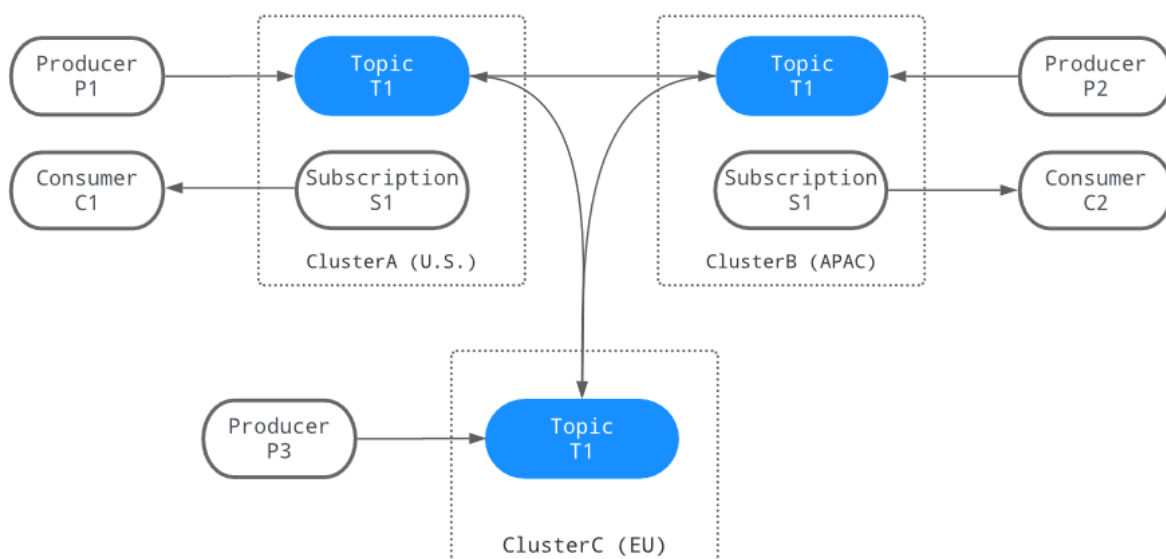
影响	集群级故障转移	GEO异地复制
管理员的工作量大吗？	• 对于自动集群级故障转移，集群切换是根据用户设置的策略自动触发的，管理员不需要进行操作。 • 对于手工集群级故障转移，切换由管理员手动触发，需要一定的工作量。	管理员工作量大。 如果集群出现故障，则需要立即进行管理干预。

影响	集群级故障转移	GEO异地复制
是否导致数据丢失？	对于自动和手工的集群级故障转移，如果发生故障的主集群没有立即将消息复制到备份集群，TongLINK/Q-CN客户端将无法使用未复制的消息。主集群恢复，TongLINK/Q-CN客户端切换回连接主集群后，非复制数据仍然可以被TongLINK/Q-CN客户端消费。因此，数据不会丢失。 • 集群级故障自动切换，业务自动切换和恢复，数据不丢失。 • 对于手工的集群级故障转移，可以手动切换和恢复服务，并且可能会发生数据丢失。	会导致数据丢失。 Pulsar客户端和DNS系统都有缓存。当管理员将DNS从主集群切换到备份集群时，缓存触发超时需要一定时间，这会延迟客户端恢复时间并无法生产或消费消息。
是否导致TongLINK/Q-CN客户端失败？	• 对于自动集群级别的故障转移，服务可以自动切换和恢复，TongLINK/Q-CN客户端不会出现故障。 • 对于手工的集群级故障转移，可以手动切换和恢复服务，但TongLINK/Q-CN客户端会在管理员采取措施之前发生故障。	同上。

3. GEO

3.1. 功能简介

GEO-Replication机制通常用于灾难恢复，支持跨多个数据中心复制持久存储的消息数据。例如，用户的应用程序在一个区域发布数据，并希望在其他区域处理这些数据以供使用。借助GEO-Replication，可以在不同的地理位置生成和使用消息。



3.2. 使用场景

- **主备集群架构：**通过GEO-Replication的数据复制能力，将主集群的数据实时备份到备用集群，当主集群发生故障不能使用，可以直接切换到备用集群进行生产消费。
- **生产和消费跨集群部署：**可以利用GEO-Replication的数据跨集群复制功能，让生产和消费分别部署在不同的集群，减少地域限制。

3.3. 配置方法

在TongLINK/Q-CN控制台可配置集群间的GEO-Replication。

1. 创建链路：使用角色为“租户管理员”的用户账号登录管理控制台，选择“复制链路 > 创建复制链路”。

链路名称	源集群	目标集群	复制类型	租户	命名空间	主题	开启链路	创建时间	操作
2-1	tonglinkq-cn-rg8...	tonglinkq-cn-g5...	命名空间	t1	ns1		☒	2025-03-24T16:5...	删除 监控
1-3	tonglinkq-cn-g5...	tonglinkq-cn-i6v...	命名空间	t1	ns1		☐	2025-03-24T16:5...	删除 监控
1-2	tonglinkq-cn-g5...	tonglinkq-cn-rg8...	命名空间	t1	ns1		☒	2025-03-25T13:3...	删除 监控

2. 在创建复制链路窗口填写必要信息，然后单击“确认”按钮。（如果需要配置双向复制，需要配置两条链路）：

创建复制链路

* 复制链路名称

请输入复制链路名称

* 源集群

请选择源集群

* 目标集群

请选择目标集群

* 复制级别

命名空间级别

* 资源租户

请选择资源租户

* 命名空间

请选择命名空间

是否启动链路

说明

0/128

取消

确认

3. 复制链路页面还可以开启/关闭链路和删除链路：

集群

集群管理

复制链路

组件日志

告警

租户

复制链路

+ 创建复制链路

请输入链路名称

搜索

链路名称	源集群	目标集群	复制级别	租户	命名空间	主题	开启链路	创建时间	操作
2-1	tonglinkq-cn-rg8...	tonglinkq-cn-g5...	命名空间	t1	ns1			2025-03-24T16:5...	删除 监控
1-3	tonglinkq-cn-g5...	tonglinkq-cn-iv...	命名空间	t1	ns1			2025-03-24T16:5...	删除 监控
1-2	tonglinkq-cn-g5...	tonglinkq-cn-rg8...	命名空间	t1	ns1			2025-03-25T13:3...	删除 监控

3.4. 业务验证

1. 创建两个集群：使用角色为“租户管理员”的用户账号登录管理控制台，选择“集群 > 集群管理 > 创建集群 > 新增实例”，在弹出的窗口中创建TongLINK/Q-CN集群。

tonglinkq-cn-g5nmvr	tonglinkq-cn-geo1	部署成功	2025-03-24 09:38:09	详情 编辑 卸载 删除
tonglinkq-cn-rg8u7m	tonglinkq-cn-geo2	部署成功	2025-03-20 15:15:53	详情 编辑 卸载 删除

2. 创建链路：使用角色为“租户管理员”的用户账号登录管理控制台，选择“复制链路 > 创建复制链路”，在弹出的窗口中创建链路。

链路名称	源集群	目标集群	复制级别	租户	命名空间	主题	开启链路	创建时间	操作
2-1	tonglinkq-cn-rg8u7m	tonglinkq-cn-g...	命名空间	t1	ns1			2025-03-24T16...	删除 监控
1-2	tonglinkq-cn-g5nmvr	tonglinkq-cn-r...	命名空间	t1	ns1			2025-03-25T13...	删除 监控

3. 分别在两个集群下创建相同的资源租户、命名空间和主题：

The screenshots show the '主题' (Topic) management interface for 't1 / ns1'. The interface includes a sidebar with '资源' (Resources) and '主题' (Topics) sections. The main area displays a table of topics. In both screenshots, the table contains one topic named 'topic1' with 0 partitions, persistent mode, 0 producers, 0 consumers, and various metrics. The creation time is 2025-04-22 13:57:52 in the left screenshot and 2025-04-22 13:55:04 in the right screenshot.

4. 创建生产者往集群1发送消息：

```
PulsarClient client = PulsarClient.builder()

    .serviceUrl("pulsar://10.10.22.171:31300")

    .build();

Producer<String> producer = client.newProducer(Schema.STRING)

    .topic("persistent://t1/ns1/topic1")

    .create();

int i = 0;

while (i < 100000) {

    String msg = "test for geo, i: " + i + ", time: " +

        LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));

    logger.info("消息发送: "+msg);

    msg = msg + new String(new byte[1024]);

    producer.send(msg);

    i++;

}
```

```
producer.close();
```

```
client.close();
```

```
2025-04-22 16:33:18.927 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 0, time: 2025-04-22 16
2025-04-22 16:33:19.040 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 1, time: 2025-04-22 16
2025-04-22 16:33:19.051 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 2, time: 2025-04-22 16
2025-04-22 16:33:19.057 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 3, time: 2025-04-22 16
2025-04-22 16:33:19.062 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 4, time: 2025-04-22 16
2025-04-22 16:33:19.067 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 5, time: 2025-04-22 16
```

5. 创建消费者在集群2消费消息:

```
PulsarClient client = PulsarClient.builder()

    .serviceUrl("pulsar://10.10.22.164:2904")

    .build();

Consumer<String> consumer = client.newConsumer(Schema.STRING)

    .topic("persistent://t1/ns1/topic1")

    .subscriptionName("subscription1")

    .subscriptionType(SubscriptionType.Shared)

    .replicateSubscriptionState(true)

    .subscribe();

while (true) {

    Message<String> message = consumer.receive();

    logger.info("是否是复制消息: " + message.isReplicated());

    logger.info("message: " + message.getValue().trim());

    logger.info("replicatedFrom: " + message.getReplicatedFrom());

    consumer.acknowledge(message);

}
```

```
2025-04-22 16:41:07.287 INFO --- [main] org.example.pulsar.ConsumerTest1 - 是否是复制消息: true
2025-04-22 16:41:07.292 INFO --- [main] org.example.pulsar.ConsumerTest1 - message: test for geo, i: 0, time: 2025-04-22 16
2025-04-22 16:41:07.292 INFO --- [main] org.example.pulsar.ConsumerTest1 - replicatedFrom: tonglinkq-cn-g5nmvr
2025-04-22 16:41:07.294 INFO --- [main] org.example.pulsar.ConsumerTest1 - 是否是复制消息: true
2025-04-22 16:41:07.294 INFO --- [main] org.example.pulsar.ConsumerTest1 - message: test for geo, i: 1, time: 2025-04-22 16
2025-04-22 16:41:07.294 INFO --- [main] org.example.pulsar.ConsumerTest1 - replicatedFrom: tonglinkq-cn-g5nmvr
2025-04-22 16:41:07.294 INFO --- [main] org.example.pulsar.ConsumerTest1 - 是否是复制消息: true
2025-04-22 16:41:07.294 INFO --- [main] org.example.pulsar.ConsumerTest1 - message: test for geo, i: 2, time: 2025-04-22 16
2025-04-22 16:41:07.296 INFO --- [main] org.example.pulsar.ConsumerTest1 - replicatedFrom: tonglinkq-cn-g5nmvr
```

可以看到集群2顺利消费消息，并且消息的isReplicated属性为true。

4. 订阅同步

4.1. 功能简介

在配置了GEO-Replication双向同步的两个跨区域部署的集群中，可以实现订阅状态同步。GEO Replication机制为订阅状态的同步提供了强大的支持。通过原生支持数据和订阅状态在多个集群之间进行复制，GEO Replication确保了数据的高可用性和灾备恢复能力。同时，通过精细的同步机制和映射转换机制，GEO Replication也保证了订阅状态在不同集群之间的一致性和准确性。

4.2. 使用场景

订阅同步的一个典型的应用常见是集群容灾，正常情况下只有主集群提供写入和消费服务，主集群故障之后，生产和消费就可以直接切换到备用集群进行生产消费。

4.3. 配置方法

要使用订阅同步功能，一定要在服务端配置双向同步的GEO功能。GEO配置可以参考[配置方法](#)。

- 在服务端，broker的配置中，需要配置参数enableReplicatedSubscriptions为true。
- 在客户端，创建订阅者的时候，需要设置replicateSubscriptionState为true。

```
Consumer<String> consumer = client.newConsumer(Schema.STRING)

    .replicateSubscriptionState(true)

    .subscribe();
```

4.4. 业务验证

1. 搭建两个集群，并配置双向GEO：

```
[lying@k8s-node-162 ~/tlqcn/TLQ-CN]$ bin/tong-admin clusters list
cluster1
cluster2
```

```
[lying@k8s-node-163 ~/tlqcn/TLQ-CN]$ bin/tong-admin clusters list
cluster1
cluster2
```

2. 在两个集群下分别创建相同的资源租户，命名空间，主题：

```
[lying@k8s-node-162 ~/tlqcn/TLQ-CN]$ bin/tong-admin topics list tenant1/ns1
persistent://tenant1/ns1/topic1
persistent://tenant1/ns1/topic2
```

```
[lying@k8s-node-163 ~/tlqcn/TLQ-CN]$ bin/tong-admin topics list tenant1/ns1
persistent://tenant1/ns1/topic1
persistent://tenant1/ns1/topic2
```

3. 在两个集群的服务端均配置订阅同步:

```
enableReplicatedSubscriptions=true
```

4. 创建生产者往cluster1发送消息:

```
PulsarClient client = PulsarClient.builder()

    .serviceUrl("pulsar://10.10.22.171:31300")

    .build();

Producer<String> producer = client.newProducer(Schema.STRING)

    .topic("persistent://t1/ns1/topic1")

    .create();

int i = 0;

while (i < 100000) {

    String msg = "test for geo, i: " + i + ", time: " +

        LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));

    logger.info("消息发送: "+msg);

    msg = msg + new String(new byte[1024]);

    producer.send(msg);

    i++;

}

producer.close();

client.close();
```

```
2025-04-22 17:26:01.886 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 0, time: 2025-04-22 17:26:01.886
2025-04-22 17:26:01.936 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 1, time: 2025-04-22 17:26:01.936
2025-04-22 17:26:04.270 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 2, time: 2025-04-22 17:26:04.270
2025-04-22 17:26:04.485 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 3, time: 2025-04-22 17:26:04.485
2025-04-22 17:26:04.488 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 4, time: 2025-04-22 17:26:04.488
2025-04-22 17:26:04.493 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 5, time: 2025-04-22 17:26:04.493
2025-04-22 17:26:04.700 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 6, time: 2025-04-22 17:26:04.700
2025-04-22 17:26:04.704 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 7, time: 2025-04-22 17:26:04.704
2025-04-22 17:26:04.707 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 8, time: 2025-04-22 17:26:04.707
2025-04-22 17:26:04.711 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 9, time: 2025-04-22 17:26:04.711
2025-04-22 17:26:04.722 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 10, time: 2025-04-22 17:26:04.722
2025-04-22 17:26:04.931 INFO --- [main] org.example.pulsar.ProducerTest - 消息发送: test for geo, i: 11, time: 2025-04-22 17:26:04.931
```

5. 创建消费者1在cluster1消费部分消息后，停止消费者1：

```
PulsarClient client = PulsarClient.builder()

    .serviceUrl("pulsar://10.10.22.162:6650")

    .build();

Consumer<String> consumer = client.newConsumer(Schema.STRING)

    .topic("persistent://t1/ns1/topic1")

    .subscriptionName("subscription1")

    .subscriptionType(SubscriptionType.Shared)

    .replicateSubscriptionState(true) //该属性需要设置为true

    .subscribe();

while (true) {

    Message<String> message = consumer.receive();

    logger.info("是否是复制消息: " + message.isReplicated());

    logger.info("message: " + message.getValue().trim());

    logger.info("replicatedFrom: " + message.getReplicatedFrom());

    consumer.acknowledge(message);

}
```

```
2025-04-22 17:26:20.912 INFO --- [main] org.example.pulsar.ConsumerTest - 是否是复制消息: false
2025-04-22 17:26:20.913 INFO --- [main] org.example.pulsar.ConsumerTest - message: test for geo, i: 197, time: 2025-04-22 1
2025-04-22 17:26:20.913 INFO --- [main] org.example.pulsar.ConsumerTest - replicatedFrom: null
2025-04-22 17:26:20.913 INFO --- [main] org.example.pulsar.ConsumerTest - 是否是复制消息: false
2025-04-22 17:26:20.913 INFO --- [main] org.example.pulsar.ConsumerTest - message: test for geo, i: 198, time: 2025-04-22 1
2025-04-22 17:26:20.913 INFO --- [main] org.example.pulsar.ConsumerTest - replicatedFrom: null
2025-04-22 17:26:22.038 INFO --- [main] org.example.pulsar.ConsumerTest - 是否是复制消息: false
2025-04-22 17:26:22.038 INFO --- [main] org.example.pulsar.ConsumerTest - message: test for geo, i: 199, time: 2025-04-22 1
2025-04-22 17:26:22.038 INFO --- [main] org.example.pulsar.ConsumerTest - replicatedFrom: null

Process finished with exit code 130
```

可以看到此时最后一条消息id为199。

6. 创建消费者2在cluster2消费消息:

```
PulsarClient client = PulsarClient.builder()

    .serviceUrl("pulsar://10.10.22.163:6650")

    .build();

Consumer<String> consumer = client.newConsumer(Schema.STRING)

    .topic("persistent://t1/ns1/topic1")

    .subscriptionName("subscription1")

    .subscriptionType(SubscriptionType.Shared)

    .replicateSubscriptionState(true) //该属性需要设置为true

    .subscribe();

while (true) {

    Message<String> message = consumer.receive();

    logger.info("是否是复制消息: " + message.isReplicated());

    logger.info("message: " + message.getValue().trim());

    logger.info("replicatedFrom: " + message.getReplicatedFrom());

    consumer.acknowledge(message);

}
```

```

2025-04-22 17:27:30.705 INFO --- [pulsar-client-io-1-6] org.apache.pulsar.client.impl.ConsumerImpl - [persistent://tena
2025-04-22 17:27:30.752 INFO --- [main] org.example.pulsar.ConsumerTest1 - 是否是复制消息: true
2025-04-22 17:27:30.754 INFO --- [main] org.example.pulsar.ConsumerTest1 - message: test for geo, i: 200, time: 2025-04-2
2025-04-22 17:27:30.754 INFO --- [main] org.example.pulsar.ConsumerTest1 - replicatedFrom: cluster1
2025-04-22 17:27:30.758 INFO --- [main] org.example.pulsar.ConsumerTest1 - 是否是复制消息: true
2025-04-22 17:27:30.758 INFO --- [main] org.example.pulsar.ConsumerTest1 - message: test for geo, i: 201, time: 2025-04-2
2025-04-22 17:27:30.759 INFO --- [main] org.example.pulsar.ConsumerTest1 - replicatedFrom: cluster1
2025-04-22 17:27:30.759 INFO --- [main] org.example.pulsar.ConsumerTest1 - 是否是复制消息: true
2025-04-22 17:27:30.759 INFO --- [main] org.example.pulsar.ConsumerTest1 - message: test for geo, i: 202, time: 2025-04-2
2025-04-22 17:27:30.759 INFO --- [main] org.example.pulsar.ConsumerTest1 - replicatedFrom: cluster1
2025-04-22 17:27:30.759 INFO --- [main] org.example.pulsar.ConsumerTest1 - 是否是复制消息: true
    
```

可以看到消费者2从id为200的消息开始消费，并且消息的isReplicated属性为true。

4.5. 局限性

在切换到备用集群的时候，由于订阅同步相比于GEO的数据同步有滞后性，所以存在一定的数据重复消费的问题，需要对这部分数据进行单独处理。

附录 A: 术语说明

名称	说明
TongLINK/Q-CN (简称TLQ-CN)	东方通云原生消息中间件软件
Broker	服务端工作节点
Client	客户端
Topic	主题
Message	消息
Producer	生产者，即消息的提供者
Consumer	消费者，即消息的消费者
Pub-Sub	发布及订阅
Point-to-Point(PTP)	点对点模式，即生产者和消费者之间的消息往来

